

How To — Ruban Rose — M1

Classification histopathologique sur le dataset BHI

Méthodologie et points difficiles

Tristan Vanrullen · La Plateforme · 2026

Objectifs Pédagogiques

1

Maîtriser le dataset BHI

Explorer la structure des fichiers, analyser la distribution des classes, calculer les statistiques de base (N=277 524 patches, ratio IDC+/IDC- ~28/72%).

2

Construire un pipeline reproductible

Implémenter un pipeline PyTorch complet et documenté : chargement → split → prétraitement → entraînement → évaluation, avec seeds fixées et résultats traçables.

3

Traiter le déséquilibre des classes

Appliquer au moins une technique compensatoire (Weighted Loss, WeightedRandomSampler ou Focal Loss) et en justifier le choix par des métriques.

4

Utiliser le transfer learning

Fine-tuner un modèle pré-entraîné (ResNet-50, EfficientNet...) avec une stratégie de dégel progressif et des learning rates différentiels.

5

Évaluer avec les bonnes métriques

Utiliser Precision, Recall, F1-score, ROC-AUC et PR-AUC en lieu et place de la seule accuracy — justifier chaque métrique dans le rapport.

Le Dataset BHI — Breast Histopathology Images

Structure du dataset

- ▶ 277 524 patches d'images (50×50 pixels, RGB)
- ▶ IDC négatif (IDC−) : 198 738 patches (~71,6 %)
- ▶ IDC positif (IDC+) : 78 786 patches (~28,4 %)
- ▶ 162 patientes distinctes (IDs numériques)
- ▶ Organisation : patient_id/0/ et patient_id/1/
- ▶ Format PNG, colorisation H&E (Hématoxyline-Éosine)

Enjeux expérimentaux clés

Déséquilibre ~3:1

IDC− vs IDC+ : l'accuracy seule est trompeuse, le modèle trivial prédit tout IDC− et atteint 71,6 %.

Patches indépendants ?

Plusieurs patches proviennent du même tissu/patient → risque de data leakage si le split n'est pas fait par patient.

Variabilité de coloration

Les colorisations H&E varient selon les labos → la normalisation de Macenko ou Reinhard est recommandée.

Interprétabilité clinique

Une bonne prédiction ne suffit pas : les cliniciens attendent une heatmap d'activation (Grad-CAM) justifiant la décision.

Pipeline Global du Projet — 7 Étapes

1	2	3	4	5	6	7
Audit du dataset	Split expérimental	Prétraitement	Augmentation	Entraînement	Évaluation	Interprétabilité
Distribution des classes, vérification des doublons, split train/val/test par patient (80/10/10).	Division stratifiée, seeds fixées (42), séparation stricte patient_id sans data leakage.	Normalisation ImageNet, resize 50→224px, normalisation de coloration H&E optionnelle.	Flips H/V, rotations aléatoires, ColorJitter, RandomErasing — uniquement sur le train set.	Transfer learning ResNet-50, weighted loss, scheduler LR, early stopping sur val_loss.	Precision, Recall, F1, ROC-AUC, PR-AUC par époque et sur le test set final.	Grad-CAM sur les faux négatifs et faux positifs, discussion clinique des cartes d'activation.

BHI = Dataset du sujet (recommandé pour le projet) :

- <https://drive.google.com/file/d/14hTFPJ2AhhgVtOXGHMjTzSGR4EvnK5AB/edit>
- ou <https://www.kaggle.com/datasets/alaminhuyan/breast-histopathology-images>

BreakHis = Dataset du Kickoff :

- <https://www.kaggle.com/datasets/ambarish/breakhis>

Défi 1 : Le Déséquilibre des Classes

Pourquoi l'accuracy seule est trompeuse ?

Le dataset BHI contient :

- 198 738 patches IDC- (~71,6 %)
- 78 786 patches IDC+ (~28,4 %)

Un modèle qui prédit toujours IDC- obtient :

- ▶ **Accuracy = 71,6 %** ← **apparemment correct !**
- ▶ **Recall IDC+ = 0 %** ← **catastrophique cliniquement**
- ▶ **F1-score IDC+ ≈ 0** ← la vraie métrique

Exemple extrême (pour illustration) :

900 bénins / 100 malins → **modèle trivial = 90%**
mais rappel malins = 0 %, tous les cancers manqués.



Conséquences cliniques des faux négatifs

Un faux négatif = cancer non détecté.

La patiente est renvoyée sans diagnostic,
ce qui peut retarder le traitement de plusieurs mois.

En pratique clinique :

- Recall (sensibilité) IDC+ doit être $\geq 0,85$
- Precision seule est insuffisante
- Le seuil de décision peut être ajusté
post-entraînement sur la courbe ROC

→ Optimiser le F1-score IDC+ ou le Recall IDC+,
pas l'accuracy globale.

Accuracy Trompeuse — Matrice de Confusion & Métriques

Matrice de confusion 2x2 (exemple chiffré, dataset déséquilibré)

	Prédit IDC-	Prédit IDC+
Réel IDC-	VN = 68 000	FP = 4 000
Réel IDC+	FN = 12 000	VP = 14 000

Formules des métriques

```
Precision = VP / (VP + FP)      = 14 000 / (14 000 + 4 000) = 0.778
Recall    = VP / (VP + FN)      = 14 000 / (14 000 + 12 000) = 0.538 ← à optimiser !
F1-score  = 2 × (P × R) / (P + R) = 0.637
Accuracy  = (VP + VN) / Total = (14 000 + 68 000) / 98 000 = 0.837 ← TROMPEUSE
F1 macro  = moyenne F1 IDC- + F1 IDC+ / 2 → meilleur indicateur global
```

► L'accuracy de 83,7 % semble bonne, mais le Recall IDC+ de 53,8 % signifie que 46 % des cancers sont manqués.

Solutions au Déséquilibre des Classes — 3 Approches

(a) Weighted Cross Entropy Loss

Principe : on assigne des poids inversement proportionnels à la fréquence de chaque classe.

$$w_c = N_{total} / (N_{classes} \times N_c)$$

Classe IDC+ : $w = 277524 / (2 \times 78786) \approx 1.76$

Classe IDC- : $w = 277524 / (2 \times 198738) \approx 0.70$

Avantage : simple à implémenter, stable.

Limite : ne rééquilibre pas les batches.

(b) WeightedRandomSampler

Principe : chaque sample est tiré avec une probabilité inversement proportionnelle à la fréquence de sa classe.

Le DataLoader tire des batches équilibrés à chaque époque → le modèle voit autant de IDC+ que de IDC-.

Avantage : rééquilibre réel des batches, compatible avec toute loss function.

Limite : sur-échantillonnage (overfitting possible).

(c) Focal Loss

$$\text{Focal Loss} = -\alpha t (1 - pt)^\gamma \times \log(pt)$$

γ (gamma) : down-weight des exemples faciles.

αt : poids par classe (comme weighted CE).

Avantage : focalise l'entraînement sur les exemples difficiles et mal classés.

Paramètres recommandés : $\gamma = 2$, $\alpha = [0.25, 0.75]$.

Limite : hyperparamètre γ à tuner.



Recommandation : combiner (a) + (b) en première approche, puis tester (c) si les résultats sont insuffisants.

Code PyTorch : Weighted Loss & WeightedRandomSampler

```
import torch
from torch.utils.data import WeightedRandomSampler, DataLoader
import torch.nn as nn

# --- 1. Calcul des poids de classes ---
# labels_train : liste/tensor des étiquettes du train set (0=IDC-, 1=IDC+)
labels_train = torch.tensor(train_dataset.targets) # ex: [0,0,1,0,1,...]

class_counts = torch.bincount(labels_train) # [N_IDC-, N_IDC+]
class_weights = 1.0 / class_counts.float() # poids inversement proportionnels
class_weights = class_weights / class_weights.sum() # normalisation (somme = 1)
# => class_weights ≈ [0.285, 0.715] (IDC+ reçoit + de poids)

# --- 2. WeightedRandomSampler : 1 poids par sample ---
sample_weights = class_weights[labels_train] # poids pour chaque image
sampler = WeightedRandomSampler(
    weights = sample_weights,
    num_samples = len(sample_weights),
    replacement = True # tirage avec remise
)

# --- 3. DataLoader avec sampler (ne pas mélanger shuffle=True et sampler) ---
train_loader = DataLoader(
    dataset = train_dataset,
    batch_size = 32,
    sampler = sampler, # remplace shuffle=True
    num_workers= 4,
    pin_memory = True
)

# --- 4. CrossEntropyLoss pondérée (renforce la pénalité sur IDC+) ---
# class_weights_loss : [w_IDC-, w_IDC+] sur GPU
class_weights_loss = torch.tensor([0.70, 1.76]).to(device)
criterion = nn.CrossEntropyLoss(weight=class_weights_loss)
```


Défi 2 : Choix des Métriques — Tableau Comparatif

Métrique	Formule	Cas d'usage	Limites (déséquilibre)
Accuracy	$VP+VN / Total$	Benchmark initial	⚠️ Trompeuse si déséquilibre. À éviter seule.
Precision	$VP / (VP+FP)$	Limiter faux positifs	Ignorée si toute prédiction = IDC-.
Recall	$VP / (VP+FN)$	★ Détecter max. cancers	Peut descendre à 0 sur modèle trivial.
F1-score	$2PR / (P+R)$	★ Équilibre P/R, déséquil.	F1 macro à préférer au F1 global.
ROC-AUC	Aire sous courbe ROC	Compare modèles global	Peut être élevé même avec mauvais Recall.
PR-AUC	Aire sous courbe P/R	★★ Meilleur sur déséquil.	Idéal : close de 1.0 sur classe rare.

★ = métriques prioritaires pour le rendu M1 | ★★ = métrique recommandée sur données médicales déséquilibrées

Défi 3 : Transfer Learning — Feature Extraction vs Fine-Tuning

Feature Extraction

(backbone gelé)

Toutes les couches conv. sont gelées (frozen).
Seule la nouvelle tête de classification est entraînée (FC layer).

Quand l'utiliser ?

- Dataset très petit (< 1 000 images)
- Similarité domaine source élevée
- Ressources GPU limitées

Entraînement rapide, faible risque d'overfitting.
Performances limitées si domaine différent.

Fine-Tuning

(backbone partiellement dégelé)

On dégèle les dernières couches du backbone
(ex: layer4 de ResNet-50) + la tête.

Quand passer en fine-tuning ?

- Après convergence de la tête (≥ 5 époques)
- Dataset suffisant (BHI : ~78k IDC+)
- LR très faible sur le backbone ($1e-5$)

Risque d'oubli catastrophique (*catastrophic forgetting*) si LR trop élevé sur le backbone.

Differential Learning Rates

(bonne pratique)

Appliquer des LR différents par groupe :

```
Backbone (gelé/pré-entraîné) : LR = 1e-5
Couches dégelées (layer4)      : LR = 5e-5
Tête de classification         : LR = 1e-3
```

Implémenté via `param_groups` dans AdamW.

Permet un fine-tuning progressif sans détruire les représentations pré-entraînées.

Code PyTorch : Fine-Tuning Progressif de ResNet-50

```
import torchvision.models as models
import torch.nn as nn

# --- 1. Chargement de ResNet-50 pré-entraîné sur ImageNet ---
model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)

# --- 2. Phase 1 : Feature Extraction - gel de TOUTES les couches ---
for param in model.parameters():
    param.requires_grad = False          # aucun gradient calculé dans le backbone

# --- 3. Remplacement de la tête de classification ---
num_features = model.fc.in_features      # 2048 pour ResNet-50
model.fc = nn.Sequential(
    nn.Dropout(p=0.5),
    nn.Linear(num_features, 256),
    nn.ReLU(),
    nn.Linear(256, 2)                    # 2 classes : IDC- et IDC+
)
# Seule la tête a requires_grad=True (nouvellement créée)

# --- 4. Phase 2 : Fine-Tuning - dégel partiel de layer4 ---
for param in model.layer4.parameters():
    param.requires_grad = True          # dégèle uniquement layer4

# --- 5. AdamW avec deux param_groups à learning rates différents ---
optimizer = torch.optim.AdamW([
    {'params': model.layer4.parameters(), 'lr': 5e-5, 'weight_decay': 1e-4}, # backbone (bas LR)
    {'params': model.fc.parameters(), 'lr': 1e-3, 'weight_decay': 1e-4},   # tête (LR normal)
])

# --- 6. Scheduler cosinus pour réduire le LR progressivement ---
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=20, eta_min=1e-6)
```

Défi 4 : Validation Expérimentale — Bonnes Pratiques

► Split propre train / val / test

Divisé UNE SEULE FOIS par patient_id (80/10/10).
Jamais de fuite entre train et test.
Stratifié selon les proportions IDC+ / IDC-.

► Seeds reproductibles

```
torch.manual_seed(42)
numpy.random.seed(42)
random.seed(42)
torch.backends.cudnn.deterministic = True
```

► Early Stopping

Surveiller val_loss (pas val_accuracy).
Patience = 7 époques sans amélioration.
Sauvegarder le meilleur checkpoint (best_model.pth).

► Logging par époque

Enregistrer train_loss, val_loss, val_f1,
val_recall_IDC+, val_precision_IDC+.
Utiliser tensorboard ou wandb pour visualiser.

► Protocole de comparaison

Toujours comparer sur le même test set.
Même seed, même split pour toutes les architectures.
Tableau comparatif obligatoire dans le rapport.

► Checklist reproductibilité

requirements.txt avec versions exactes
config.yaml pour tous les hyperparamètres
README.md avec instructions d'exécution

Interprétabilité avec Grad-CAM

Principe de Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) calcule l'importance de chaque région spatiale de la feature map pour la prédiction finale.

Gradient du score de classe cible $\rightarrow$ couche conv. cible
 $\partial y^c / \partial A^k$ (gradient par rapport à chaque canal k)
Poids $\alpha^c_k = (1/Z) \sum_{i,j} \text{gradient}$

La heatmap : $L^c_{\text{Grad-CAM}} = \text{ReLU}(\sum_k \alpha^c_k A^k)$

Application : pytorch-grad-cam (bibliothèque recommandée)
`pip install grad-cam`
`from pytorch_grad_cam import GradCAM`
`from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget`

Code d'usage (pytorch-grad-cam)

```
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image

# Cibler la dernière couche conv. (layer4[-1] pour ResNet-50)
cam = GradCAM(model=model, target_layers=[model.layer4[-1]])

# Générer la heatmap pour une image
targets = [ClassifierOutputTarget(1)] # classe IDC+
grayscale_cam = cam(input_tensor=img_tensor, targets=targets)

# Superposer sur l'image originale
visualization = show_cam_on_image(img_rgb, grayscale_cam[0])
```

Interprétation de la heatmap H&E

- Zone rouge = région qui a le plus contribué à la prédiction IDC+.
- Sur une image H&E : le modèle devrait activer sur les noyaux cellulaires denses (zones sombres, irrégulières).
- Comparer les activations entre vrais positifs et faux positifs pour identifier les biais.

Bonne prédiction $\neq$ Bonne justification

Un modèle peut prédire IDC+ correctement en se basant sur des artefacts de numérisation (bruit, flou de bord) plutôt que sur les caractéristiques biologiques réelles.

Analyser 5+ images de faux négatifs avec Grad-CAM et commenter les zones activées dans le rapport.

Checklist Projet — Ce que le Rendu M1 Doit Contenir

✓ Obligatoire — 7 critères

Audit du dataset documenté

Distribution classes, stats patches, vérification split par patient, visualisation de 20+ images.

Traitement du déséquilibre

Au moins une technique appliquée (Weighted Loss ou Sampler), comparée à une baseline sans pondération.

Métriques pertinentes

Precision, Recall, F1 IDC+ et PR-AUC reportées pour chaque modèle testé. Pas d'accuracy seule.

Fine-tuning documenté

Architecture choisie, justification du dégel, learning rates différentiels, courbes train/val.

Validation propre

Split unique par patient, seeds fixées, early stopping, test set évalué une seule fois en fin.

Grad-CAM inclus

Au moins 5 visualisations Grad-CAM commentées (vrais positifs + faux négatifs).

Discussion des erreurs

Analyse de la matrice de confusion, types d'erreurs, pistes d'amélioration argumentées.

★ Bonus — Pour aller plus loin

Focal Loss implémentée

Comparaison Focal Loss vs Weighted CE sur les mêmes métriques.

Normalisation couleur H&E

Normalisation de Macenko ou Reinhard appliquée, impact mesuré.

Expérience multi-architectures

EfficientNet-B0, DenseNet-121 ou ViT comparés à ResNet-50.

Ablation study

Comparaison feature extraction vs fine-tuning, avec/sans augmentation.

Rapport reproductible

Notebook auto-exécutable, requirements.txt, config.yaml, README.